

Berliner Hochschule für Technik

Bachelorarbeit

Konzeption und Implementierung einer interaktiven Visualisierung präemptiver Scheduling-Verfahren für die Lehre

Marten Kirschenhofer
Matrikelnummer: 833825
Studiengang: Medieninformatik

Betreuung: Prof. Dr. Christian Forler
Abgabedatum: 15.09.2026

Vorwort

Diese Bachelorarbeit entstand im Studiengang Medieninformatik an der Berliner Hochschule für Technik. Sie behandelt die Konzeption und Implementierung einer interaktiven Anwendung zur Veranschaulichung präemptiver CPU-Scheduling-Verfahren.

Danksagung

Ich widme diese Arbeit meinen lieben Schwiegereltern, die mir stets mit gutem Rat zur Seite standen, meiner wundervollen Verlobten, die mich auch in schwierigen Zeiten unterstützt hat, und meinem besten Freund, bei dem ich trotz des großen Stresses stets ich selbst sein konnte.

Inhaltsverzeichnis

Vorwort	ii
Abbildungsverzeichnis	vi
Tabellenverzeichnis	vii
Verzeichnis der Listings	viii
1. Einleitung	1
1.1. Motivation und Problemstellung	1
1.2. Zielsetzung der Arbeit	1
1.3. Thematische Ausrichtung und Abgrenzung	2
1.4. Vorgehensweise und Aufbau der Arbeit	2
1.5. Leitgedanke der Arbeit	3
2. Grundlagen	4
2.1. Betriebssysteme und Prozessverwaltung	4
2.2. CPU-Scheduling in Betriebssystemen	4
2.3. Präemptive versus nicht-präemptive Scheduling-Verfahren	5
2.4. Überblick über die betrachteten Algorithmen	6
2.5. Kriterien zur Bewertung von Scheduling-Verfahren	7
2.6. Stand der Forschung zu Scheduling-Visualisierungen	7
3. Anforderungsanalyse	8
3.1. Zielgruppe und Einsatzkontext	8
3.2. Fachliche Anforderungen	8
3.3. Didaktische Anforderungen	8
3.4. Interaktions- und Usability-Anforderungen	9
3.5. Anforderungen an Accessibility	9
3.6. Technische Qualitätsziele	9
3.7. Abgeleitete funktionale und nicht-funktionale Anforderungen	10
4. Systementwurf	11
4.1. Entwurfsziele und Leitprinzipien	11
4.2. Gesamtarchitektur und Schichtenmodell	11
4.3. Komponenten und Verantwortlichkeiten	12
4.4. Datenfluss und Zustandsmodell	13
4.5. Interaktionsmodell für den Lernprozess	14
4.6. HCI-Leitlinien im Entwurf	14
4.7. Erweiterbarkeit und Entwurfsgrenzen	14
5. Implementierung	16
5.1. Technische Grundlagen und verwendete Technologien	16

5.2. Datenmodell und Simulationskern	16
5.3. Umsetzung der Scheduling-Verfahren	17
5.4. Umsetzung der Visualisierung	17
5.5. Interaktionsmechanismen und Nutzerworkflow	17
5.6. Build- und Deploymentprozess	18
5.7. Besondere technische Herausforderungen	18
6. Qualitätssicherung	19
6.1. Qualitätsziele und Prüfstrategie	19
6.2. Automatisierte Verifikation der Simulationslogik	19
6.3. Prüfung von Interaktion und Accessibility	20
6.4. Reproduzierbarkeit und Vergleichbarkeit	20
6.5. Erreichte Testabdeckung und Grenzen	20
6.6. Fazit zur Qualitätssicherung	20
7. Ergebnis und Produktpräsentation	22
7.1. Entwickeltes Endprodukt	22
7.2. Produktüberblick und Nutzerfluss	22
7.3. Produktpräsentation mit Screenshots	23
7.4. Umgesetzte Funktionalität im Soll-Ist-Abgleich	23
7.5. Ergebnisbetrachtung	25
8. Fazit und Ausblick	26
8.1. Ausblick	26
Literaturverzeichnis	27
A. Anhang	28
A.1. Ergänzende Abbildungen	28
A.2. Zusätzliche Testfälle und Szenarien	28
A.3. Ergänzende technische Artefakte (z. B. Konfigurationsbeispiele)	28

Abbildungsverzeichnis

4.1. Komponentenmodell der Anwendung mit den Ebenen Benutzeroberfläche, Anwendungslogik und Visualisierung.	12
4.2. Übersicht der Benutzeroberfläche: Eingabebereich, Simulationsfenster und Metriken-Anzeige.	13
7.1. Startansicht der Anwendung mit Ausgangszustand und Einstieg in die Szenariokonfiguration.	23
7.2. Geladenes Szenario mit sichtbaren Task-Parametern als Grundlage für die Simulation.	23
7.3. Fokusansicht mit zeitlicher Ablaufdarstellung und Kennzahlenbereich für einen einzelnen Lauf.	24
7.4. Detaildarstellung der Gantt-Ansicht mit segmentierter Ausführung und sichtbaren Task-Wechseln.	24
7.5. Warteschlangen- und Statusdarstellung zur Einordnung der aktuellen Scheduling-Entscheidung.	24
7.6. Kennzahlenbereich mit Wartezeit, Durchlaufzeit und Reaktionszeit als quantitative Auswertung des Laufs.	24
7.7. Vergleichsansicht mehrerer Verfahren auf identischer Szenariobasis.	25

Tabellenverzeichnis

Verzeichnis der Listings

1. Einleitung

1.1. Motivation und Problemstellung

Scheduling-Verfahren gehören zu den zentralen Konzepten moderner Betriebssysteme. Sie bestimmen, welcher Prozess zu welchem Zeitpunkt CPU-Zeit erhält, und beeinflussen unmittelbar Reaktionsfähigkeit, Fairness und Auslastung eines Systems. Obwohl diese Verfahren in Lehrbüchern und Vorlesungen theoretisch sauber beschrieben werden, fällt es Studierenden häufig schwer, ihren zeitlichen Ablauf in konkreten Situationen zu verstehen Stallings [2003], Tanenbaum and Bos [2016].

Gerade präemptive Verfahren sind für Lernende anspruchsvoll, weil sie viele Zustandswechsel, Unterbrechungen und Neuplanungen erzeugen. Die Entscheidungslogik bleibt bei reinen Textbeschreibungen und statischen Beispielen oft abstrakt. Das Problem zeigt sich besonders beim Vergleich verschiedener Verfahren unter identischen Eingaben: Die fachlichen Unterschiede sind zwar theoretisch bekannt, aber in ihrer unmittelbaren Wirkung auf Ablauf, Wartezeiten und Reaktionsverhalten nicht immer intuitiv nachvollziehbar.

Vor diesem Hintergrund wurde eine interaktive Webanwendung entwickelt, die Scheduling-Verfahren als sichtbaren, steuerbaren Ablauf modelliert. Ziel ist nicht die Beantwortung einer rein wissenschaftlichen Forschungsfrage, sondern die Erstellung eines nutzbaren didaktischen Werkzeugs, das technische Zusammenhänge transparent macht und Studierenden einen praktischen Zugang zu den Grundlagen des CPU-Scheduling eröffnet.

1.2. Zielsetzung der Arbeit

Die vorliegende Arbeit verfolgt die Konzeption und Umsetzung einer interaktiven Webanwendung zur Veranschaulichung präemptiver Scheduling-Verfahren. Ziel ist die Entwicklung einer Anwendung, die:

- ausgewählte Scheduling-Verfahren fachlich korrekt simuliert,
- ihre zeitlichen Abläufe verständlich visualisiert,
- verschiedene Verfahren anhand identischer Szenarien vergleichbar macht,
- und als didaktisches Werkzeug im Lehrkontext nutzbar ist.

Der Fokus liegt damit auf der praktischen Umsetzung eines verständlichen und erweiter-

baren Produkts. Die Arbeit verbindet fachliche Grundlagen aus der Betriebssystemtheorie mit Prinzipien aus Software Engineering und Human-Computer Interaction. Im Vordergrund stehen eine nachvollziehbare Architektur, klare Interaktion und ein belastbares Produkt für den Einsatz im Studium.

1.3. Thematische Ausrichtung und Abgrenzung

Die Arbeit ist bewusst nicht als klassische empirische Untersuchung konzipiert, sondern als engineering-orientierte Produktdokumentation. Der Schwerpunkt liegt auf der Gestaltung einer Anwendung, die im Studienkontext einen Mehrwert bietet und zugleich technische, didaktische und gestalterische Entscheidungen nachvollziehbar dokumentiert.

Diese Ausrichtung ist für den Studiengang geeignet, weil sie die Verbindung zwischen theoretischem Wissen und praktischer Umsetzung sichtbar macht. Die fachlichen Grundlagen aus der Betriebssystemlehre werden mit einer strukturierten Architektur, benutzerfreundlicher Oberfläche und konkreten Produktbeispielen verknüpft. Dadurch entsteht eine Bachelorarbeit, die nicht nur theoretisch fundiert ist, sondern auch die Entwicklung eines nutzbaren Softwaresystems als zentrale Leistung hervorhebt.

Die Arbeit grenzt sich dabei von einer empirischen Wirksamkeitsstudie mit Nutzergruppen, statistischer Auswertung und hypothesengetriebener Evaluation ab. Stattdessen liegt der Fokus auf fachlicher Korrektheit der Simulation, nachvollziehbarer Architektur, technischer Umsetzung und qualitativ begründeter Produktpräsentation.

1.4. Vorgehensweise und Aufbau der Arbeit

Die Arbeit folgt einem engineering-orientierten Vorgehen:

1. Analyse der fachlichen Grundlagen zu CPU-Scheduling und relevanten Messgrößen,
2. Ableitung von Anforderungen an eine didaktische Visualisierungsanwendung,
3. Entwurf der Systemarchitektur und des Interaktionsmodells,
4. technische Umsetzung der Webanwendung,
5. Qualitätssicherung der Simulationslogik und Interaktion,
6. produktorientierte Darstellung und Einordnung der Ergebnisse.

Der Aufbau der Bachelorarbeit folgt dieser Vorgehensweise: Kapitel 2 behandelt die fachlichen Grundlagen. Kapitel 3 leitet daraus die Anforderungen ab. Kapitel 4 beschreibt den Systementwurf mit Architektur- und Interaktionsmodell. Kapitel 5 dokumentiert die technische Implementierung. Kapitel 6 stellt die Maßnahmen zur Qualitätssicherung dar. Kapitel 7 präsentiert das entwickelte Produkt und ordnet die Ergebnisse ein. Kapitel 8 schließt mit Fazit und Ausblick.

1.5. Leitgedanke der Arbeit

Der zentrale Gedanke dieser Arbeit lautet:

Praktische Verständlichkeit entsteht nicht nur durch fachliche Korrektheit, sondern auch durch eine klar strukturierte Architektur, nachvollziehbare Interaktion und eine verständliche Darstellung des zeitlichen Ablaufs.

Diese Perspektive bildet den Kern der Arbeit. Sie macht deutlich, dass die Simulation von Scheduling-Verfahren nicht nur eine fachliche Aufgabe ist, sondern auch ein Entwurfs- und Gestaltungsproblem mit unmittelbaren Auswirkungen auf Nutzbarkeit und Lernwirkung.

2. Grundlagen

Dieses Kapitel beschreibt die theoretischen Grundlagen, auf denen Konzeption, Implementierung und Bewertung der Anwendung aufbauen.

2.1. Betriebssysteme und Prozessverwaltung

Betriebssysteme koordinieren den Zugriff auf Hardware-Ressourcen und stellen Ausführungsumgebungen für Programme bereit. Eine Kernaufgabe ist dabei die Verwaltung konkurrierender Ausführungseinheiten, die um CPU-Zeit, Speicher und weitere Ressourcen konkurrieren Stallings [2003], Tanenbaum and Bos [2016].

Historisch stand der Begriff des Prozesses im Mittelpunkt: Ein Prozess ist eine vollständige, isolierte Ausführungsumgebung mit eigenem Adressraum, Dateihandles und Ressourcenzustand. Prozesse durchlaufen typischerweise mehrere Zustände, etwa bereit, laufend und blockiert. Das Betriebssystem steuert Übergänge zwischen diesen Zuständen und bestimmt durch Scheduling-Entscheidungen, welcher bereite Prozess als Nächstes ausgeführt wird. Diese Entscheidungen wirken sich direkt auf wahrgenommene Systemreaktion und Gesamtdurchlauf aus.

Parallel zur Prozessabstraktion existiert das Konzept des Threads (oder leichtgewichtigen Prozesses). Ein Thread ist eine Ausführungseinheit innerhalb eines Prozesses und teilt sich Speicher, offene Dateien und andere Ressourcen mit anderen Threads desselben Prozesses. Der Hauptunterschied liegt in Isolation und Ressourcenaufwand: Ein Kontextwechsel zwischen Threads ist in der Regel schneller als zwischen Prozessen, da weniger Systemzustand gesichert und wiederhergestellt werden muss.

Aus der Perspektive des CPU-Schedulers gibt es zwei Ebenen der Entscheidung: Auf Kernel-Ebene plant der Scheduler Prozesse oder Kernel-Threads ein, während auf Benutzerebene zusätzlich anwendungsspezifische Strategien für User-Threads existieren können. Für die vorliegende Arbeit bleibt die Prozessabstraktion als didaktische Vereinfachung ausreichend, da die grundlegenden Scheduling-Prinzipien identisch bleiben.

2.2. CPU-Scheduling in Betriebssystemen

CPU-Scheduling beschreibt die Auswahlstrategie für die Vergabe von Prozessorzeit an bereite Ausführungseinheiten. Ziel ist ein sinnvoller Ausgleich zwischen konkurrierenden

Kriterien wie kurzer Reaktionszeit, niedriger Wartezeit, hoher Auslastung und fairer Behandlung Stallings [2003].

In Mehrprogrammsystemen ist Scheduling eine kontinuierliche Entscheidungsaufgabe unter Unsicherheit, da Ankunftszeitpunkte und Laufzeitverhalten variieren. Deshalb sind Scheduling-Verfahren typischerweise als Heuristiken formuliert, deren Eignung vom Lastprofil und vom Systemziel abhängt. Unabhängig davon, ob Prozesse oder Threads geplant werden, bleiben die zentralen Fragen gleich: Auswahl des nächsten Elements, Behandlung von Präemption und Kontextwechsel sowie Auswertung über Kennzahlen wie Warte- und Durchlaufzeit.

2.3. Präemptive versus nicht-präemptive Scheduling-Verfahren

Eine grundlegende Klassifikation unterscheidet zwischen präemptiven und nicht-präemptiven Strategien. Dieser Unterschied beeinflusst sowohl das Systemverhalten als auch die Komplexität von Analyse und Visualisierung.

2.3.1. Nicht-präemptive Verfahren

Nicht-präemptive Verfahren erlauben es einem Prozess, die CPU bis zum Abschluss oder bis zur freiwilligen Blockierung zu behalten. Scheduling-Entscheidungen treten damit nur an diskreten Zeitpunkten auf. Vorteilhaft sind geringerer Kontextwechsel-Overhead und eine vergleichsweise einfache Implementierung. Nachteilig ist die Gefahr langer Wartezeiten für nachfolgende Prozesse.

Klassische Beispiele sind FCFS und SJF in nicht-präemptiver Form.

2.3.2. Präemptive Verfahren

Präemptive Verfahren erlauben dem Scheduler, einen laufenden Prozess aktiv zu unterbrechen und einen anderen Prozess einzuplanen. Auslöser sind typischerweise Zeitquanten (Timer-Interrupt) oder die Ankunft eines Prozesses mit höherer Priorität.

Vorteile liegen in besserer Reaktionsfähigkeit und höherer Steuerbarkeit, insbesondere in interaktiven Systemen. Gleichzeitig erhöhen Präemptionen die Zahl der Kontextwechsel und damit den Verwaltungsaufwand Tanenbaum and Bos [2016].

2.3.3. Herausforderungen präemptiver Verfahren

Die Analyse präemptiver Verfahren ist anspruchsvoll, weil Prozesse mehrfach unterbrochen und später fortgesetzt werden. Dadurch entstehen dynamische, nicht-lineare Abläufe, deren Auswirkungen auf Kennzahlen nicht unmittelbar offensichtlich sind. Genau an dieser Stelle

bieten interaktive Visualisierungen einen didaktischen Mehrwert, da sie Entscheidungen und Konsequenzen zeitlich nachvollziehbar machen.

Die vorliegende Anwendung konzentriert sich daher bewusst auf präemptive Verfahren.

2.4. Überblick über die betrachteten Algorithmen

Alle in der Anwendung implementierten Verfahren sind präemptiv und bilden unterschiedliche Strategien zur Auswahl des nächsten Prozesses ab.

2.4.1. Last Come First Served (LCFS)

In der präemptiven Variante von LCFS verdrängen neu eintreffende Prozesse den aktuell laufenden Prozess unmittelbar. Das Verfahren verdeutlicht Präemptionsdynamik besonders klar, kann aber unter ungünstigen Lastprofilen zu langen Wartezeiten früher Prozesse führen.

2.4.2. Round Robin

Round Robin weist jedem Prozess ein Zeitquantum zu. Nach Ablauf des Quantums wird präemptiert und der Prozess am Ende der Bereitwarteschlange wieder eingereiht. Das Verfahren gilt als faires Basismodell für zeitscheibenbasierte Planung und ist didaktisch gut nachvollziehbar Stallings [2003].

2.4.3. Strict Priority

Strict Priority wählt stets den bereiten Prozess mit der höchsten Priorität und führt sofortige Präemption bei höher priorisierten Neuankünften durch. Das Verfahren verbessert die Reaktion auf wichtige Aufgaben, kann jedoch zu Benachteiligung niedriger Prioritäten führen Tanenbaum and Bos [2016].

2.4.4. Multilevel Feedback Queue (MLFQ)

MLFQ kombiniert mehrere Warteschlangen mit unterschiedlichen Prioritäten und Zeitquanten. Prozesse starten in hoher Priorität und werden bei CPU-intensivem Verhalten schrittweise herabgestuft. Damit entsteht ein adaptives Verfahren, das kurze/interaktive und lange CPU-lastige Aufgaben unterschiedlich behandelt Meing and Tandingan [2025].

2.5. Kriterien zur Bewertung von Scheduling-Verfahren

Die Bewertung erfolgt über definierte Kennzahlen, um Verfahren nicht nur qualitativ, sondern auch quantitativ zu vergleichen Stallings [2003].

2.5.1. Wartezeit

Die Wartezeit ist die Zeitspanne, die ein Prozess im Zustand bereit verbringt, ohne ausgeführt zu werden.

2.5.2. Durchlaufzeit

Die Durchlaufzeit (Turnaround Time) ist die Zeit vom Eintreffen bis zum Abschluss eines Prozesses.

2.5.3. Reaktionszeit

Die Reaktionszeit beschreibt die Zeit vom Eintreffen bis zur ersten CPU-Zuteilung und ist besonders relevant für interaktive Systeme Tanenbaum and Bos [2016].

Ergänzend können Kontextwechselhäufigkeit, Präemptionsanzahl und Leerlaufanteile betrachtet werden, um Zielkonflikte zwischen Responsivität, Durchsatz und Ressourceneffizienz sichtbar zu machen.

2.6. Stand der Forschung zu Scheduling-Visualisierungen

Die Forschung zur Algorithmusvisualisierung zeigt, dass interaktive Darstellungen Lernprozesse unterstützen können, insbesondere wenn Zustandswechsel und zeitliche Dynamik nachvollziehbar dargestellt werden Hundhausen et al. [2002].

Für präemptive Scheduling-Verfahren ist diese didaktische Herausforderung besonders hoch, da häufige Unterbrechungen und Wiedereinplanungen zu komplexen Ablaufmustern führen. Interaktive Visualisierung kann hier helfen, Entscheidungspfade verständlich zu machen und die Wirkung einzelner Strategien auf zentrale Kennzahlen transparent darzustellen.

Die vorliegende Anwendung positioniert sich in diesem Kontext als didaktisches Werkzeug, das fachlich korrekte Simulation mit verständlicher, interaktiver Darstellung verbindet.

3. Anforderungsanalyse

Die fachlichen Anforderungen an die Anwendung ergeben sich aus der Kombination von Betriebssystemtheorie, didaktischen Zielen und softwaretechnischen Qualitätsansprüchen. Ziel ist nicht nur die korrekte Abbildung präemptiver Scheduling-Verfahren, sondern deren verständliche und steuerbare Darstellung für Lernende.

3.1. Zielgruppe und Einsatzkontext

Die primäre Zielgruppe sind Studierende in einführenden und vertiefenden Lehrveranstaltungen zu Betriebssystemen. Die Anwendung soll helfen, zeitliche Entscheidungen bei präemptiven Verfahren nachvollziehbar zu machen. Der Einsatz reicht von der begleiteten Demonstration in der Lehre bis zur eigenständigen Exploration einzelner Szenarien.

3.2. Fachliche Anforderungen

Aus fachlicher Sicht muss die Anwendung die Verfahren Round Robin, LCFS, Strict Priority und MLFQ konsistent simulieren. Dafür müssen zentrale Eingabeparameter wie Ankunftszeit, Burst-Zeit, Priorität und Zeitquantum korrekt verarbeitet werden. Zusätzlich müssen relevante Ergebnisgrößen bereitgestellt werden, insbesondere Wartezeit, Durchlaufzeit, Reaktionszeit, Kontextwechsel und Präemptionsanzahl.

Eine zentrale Anforderung lautet daher: Simulationslogik, Ereignisverlauf und Kennzahlen müssen aus derselben Zustandsbasis entstehen. Nur so bleiben die angezeigten Ergebnisse fachlich konsistent und vergleichbar.

3.3. Didaktische Anforderungen

Die didaktische Qualität ist für den Projekterfolg wesentlich. Lernende sollen nicht nur Endergebnisse sehen, sondern auch verstehen, warum ein Prozess zu einem bestimmten Zeitpunkt ausgewählt, unterbrochen oder zurückgestellt wird. Zustandswechsel müssen daher sichtbar und nachvollziehbar sein.

Drei Leitprinzipien stehen im Mittelpunkt:

- **Transparenz:** Dispatch, Präemption, Kontextwechsel und Abschluss müssen klar erkennbar sein.
- **Vergleichbarkeit:** Mehrere Verfahren sollen unter identischer Ausgangsbasis gegenübergestellt werden können.
- **Kontrollierbares Lerntempo:** Sowohl schrittweise Wiedergabe als auch kontinuierliches Abspielen müssen unterstützt werden.

3.4. Interaktions- und Usability-Anforderungen

Die Oberfläche soll die kognitive Last reduzieren und den typischen Arbeitsablauf unterstützen: Szenario definieren, Verfahren auswählen, Lauf starten, Ergebnis interpretieren, Verfahren vergleichen. Funktionen müssen in nachvollziehbarer Reihenfolge verfügbar sein und ohne unnötige Bedienhürden nutzbar bleiben.

Für den didaktischen Einsatz ist zusätzlich entscheidend, dass relevante Zustände direkt sichtbar sind, Interaktionen konsistent ausfallen und die Bedienlogik nicht von der fachlichen Analyse ablenkt.

3.5. Anforderungen an Accessibility

Da die Anwendung im Lehrkontext eingesetzt wird, sind grundlegende Accessibility-Anforderungen zu berücksichtigen. Dazu gehören eine semantisch klare Struktur, nachvollziehbare Fokussführung, verständliche Statuskommunikation und ausreichende visuelle Kontraste. Ziel ist, dass zentrale Lern- und Bedienfunktionen auch ohne ausschließlich visuelle Orientierung nutzbar bleiben W3C [2018].

3.6. Technische Qualitätsziele

Neben fachlicher Korrektheit und didaktischer Verständlichkeit sind technische Qualitätsziele zentral. Die Anwendung soll modular, deterministisch und erweiterbar aufgebaut sein. Eine klare Trennung zwischen Simulationskern, Zustandsverwaltung und Visualisierung reduziert Fehleranfälligkeit und erleichtert Erweiterungen, etwa durch zusätzliche Algorithmen oder weitere Analyseansichten.

Diese Qualitätsziele sichern nicht nur die Wartbarkeit des Systems, sondern stützen unmittelbar die fachliche Verlässlichkeit und didaktische Nutzbarkeit.

3.7. Abgeleitete funktionale und nicht-funktionale Anforderungen

Aus den vorherigen Abschnitten lassen sich die Kernanforderungen bündeln.

Funktionale Anforderungen

- Erfassung und Bearbeitung von Prozessen mit relevanten Scheduling-Parametern.
- Simulation der Verfahren Round Robin, LCFS, Strict Priority und MLFQ.
- Visualisierung des zeitlichen Ablaufs inklusive Präemptionen und Zustandswechseln.
- Ausgabe zentraler Kennzahlen pro Lauf.
- Vergleich mehrerer Läufe auf identischer Szenariobasis.
- Interaktive Ablaufsteuerung mit Start, Pause, Schrittmodus und Zeitsprung.

Nicht-funktionale Anforderungen

- Deterministisches Verhalten bei identischer Eingabe.
- Klare Trennung von Fachlogik und Darstellung.
- Verständliche, konsistente und zugängliche Benutzerführung.
- Erweiterbarkeit für zusätzliche Verfahren und Darstellungsformen.
- Testbarkeit der Kernlogik und zentraler Interaktionspfade.

Die abgeleiteten Anforderungen bilden die Grundlage für den Systementwurf im nächsten Kapitel.

4. Systementwurf

Die Anwendung basiert auf der Leitidee, dass fachliche Korrektheit und nachvollziehbare Interaktion gemeinsam gedacht werden müssen. Ein Scheduling-Visualizer soll nicht nur Ergebnisse berechnen, sondern auch verständlich machen, warum ein Prozess zu einem bestimmten Zeitpunkt läuft und ein anderer wartet. Der Systementwurf verbindet daher Architekturentscheidungen mit didaktischen Anforderungen.

4.1. Entwurfsziele und Leitprinzipien

Aus der Anforderungsanalyse ergeben sich vier zentrale Entwurfsziele:

- **Fachliche Konsistenz:** Simulation, Ereignisverlauf und Kennzahlen müssen aus einer gemeinsamen Zustandsbasis abgeleitet werden.
- **Didaktische Nachvollziehbarkeit:** Zustandswechsel und Präemptionen müssen zeitlich verständlich dargestellt werden.
- **Deterministik:** Identische Eingaben sollen zu identischen Abläufen führen.
- **Erweiterbarkeit:** Neue Algorithmen und Darstellungsformen sollen ohne grundlegenden Umbau integrierbar sein.

Diese Ziele werden durch eine komponentenbasierte Struktur mit klaren Verantwortlichkeiten umgesetzt. Die Kopplung zwischen Simulationslogik und Darstellung bleibt bewusst gering, damit fachliche Änderungen kontrolliert möglich sind und die UI als Vermittlungsschicht wirken kann.

4.2. Gesamtarchitektur und Schichtenmodell

Das System ist in drei logisch getrennte Ebenen gegliedert:

- **Szenario- und Konfigurationslogik:** Verwaltung von Taskdaten, Parametern und Laufkonfigurationen.
- **Simulationskern:** Deterministische Berechnung des zeitlichen Ablaufs je Scheduling-Verfahren.
- **Darstellung und Steuerung:** Visualisierung der Ergebnisse sowie Interaktion (Start, Pause, Schrittmodus, Zeitnavigation, Vergleich).

Die Steuerung ist dabei Teil der Darstellungsschicht, weil sie den didaktischen Ablauf unmittelbar beeinflusst: Nutzende greifen über die Oberfläche in die Wiedergabe ein und erhalten direkt visuelles Feedback auf denselben fachlichen Daten.

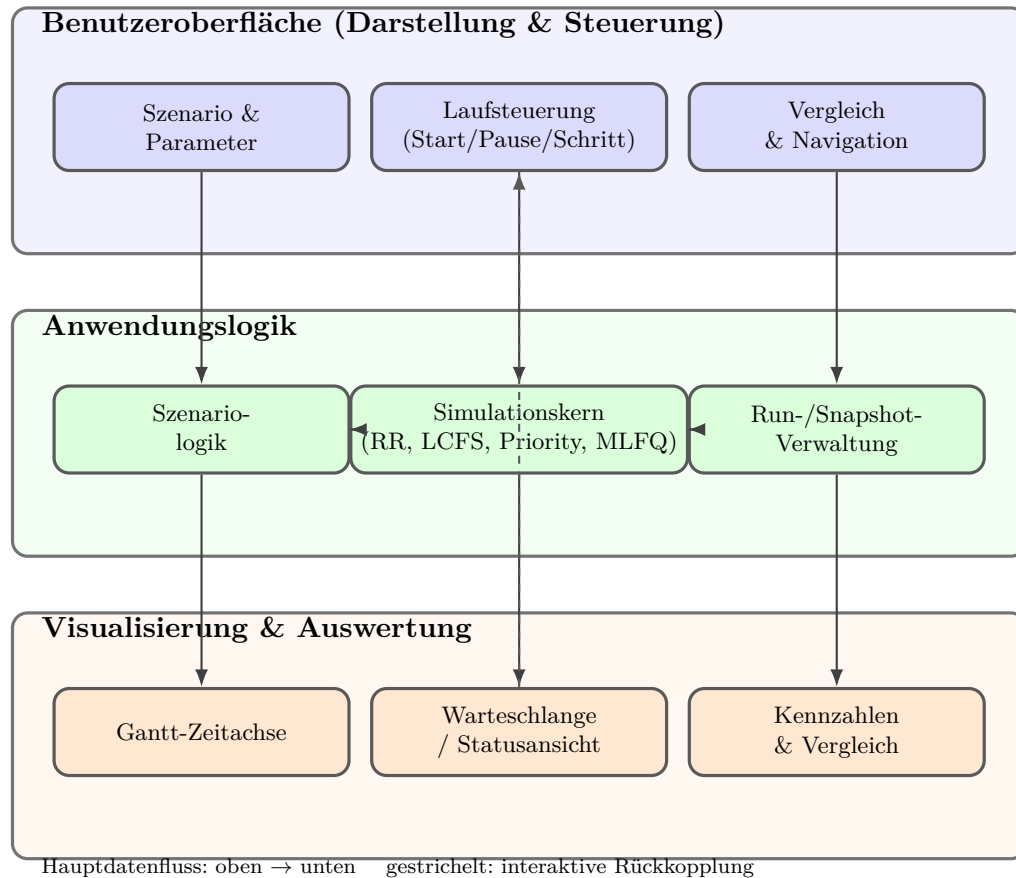


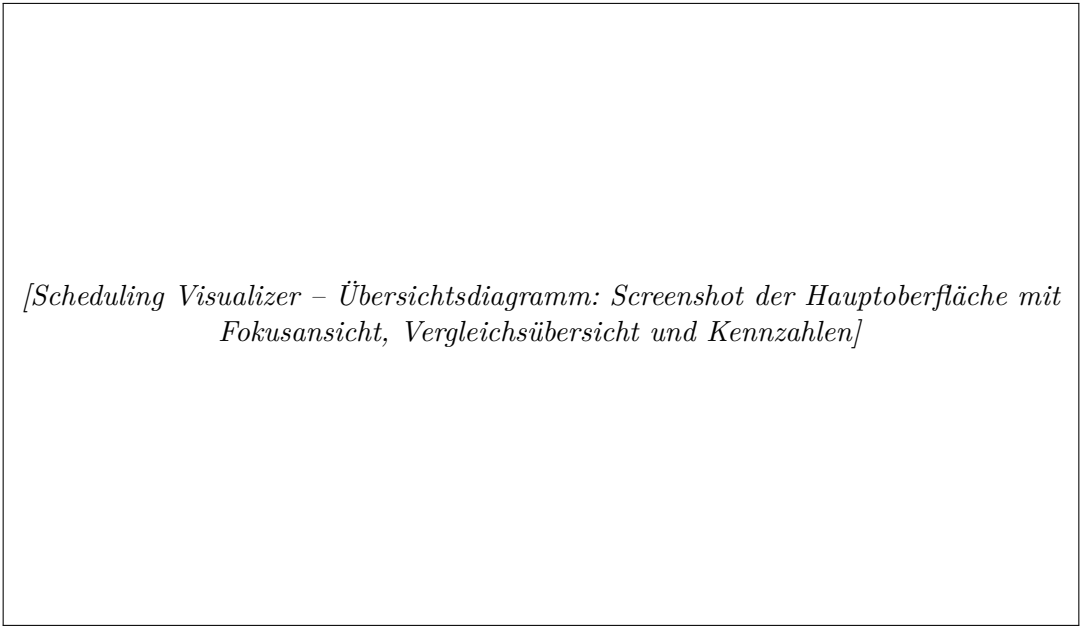
Abbildung 4.1.: Komponentenmodell der Anwendung mit den Ebenen Benutzeroberfläche, Anwendungslogik und Visualisierung.

Die Pfeile in Abbildung 4.1 repräsentieren den Datenfluss vom Eingabeszenario über die Simulationsberechnung zur Visualisierung. Steueraktionen lösen keine fachlich getrennte Parallelwelt aus, sondern wirken auf denselben konsistenten Ablaufzustand.

4.3. Komponenten und Verantwortlichkeiten

4.3.1. Szenario- und Konfigurationskomponente

Diese Komponente verwaltet die Eingabedaten der Prozesse (z. B. Ankunftszeit, Burst-Zeit, Priorität) sowie algorithmusspezifische Parameter (z. B. Zeitquantum). Zusätzlich verantwortet sie Validierung und Normalisierung der Eingaben, sodass alle Läufe auf strukturell konsistenten Daten aufsetzen.



[Scheduling Visualizer – Übersichtsdiagramm: Screenshot der Hauptoberfläche mit Fokusansicht, Vergleichsübersicht und Kennzahlen]

Abbildung 4.2.: Übersicht der Benutzeroberfläche: Eingabebereich, Simulationsfenster und Metriken-Anzeige.

4.3.2. Simulationskomponente

Die Simulationskomponente bildet den fachlichen Kern. Sie erzeugt aus Szenario und Verfahrensdefinition den vollständigen Ablauf als deterministische Folge von Zustandsübergängen. Ergebnisse werden nicht nur als Endzustand abgelegt, sondern in auswertbaren Strukturen bereitgestellt (Segmente, Ereignisse, Kennzahlen).

4.3.3. Visualisierungs- und Interaktionskomponente

Diese Komponente übersetzt Simulationsdaten in verständliche Ansichten: zeitlicher Ablauf (Gantt), Warteschlangen-/Statusdarstellung und Kennzahlen. Gleichzeitig steuert sie die Wiedergabe über Interaktionsfunktionen und ermöglicht den Vergleich mehrerer Läufe auf identischer Datenbasis.

4.4. Datenfluss und Zustandsmodell

Der Datenfluss folgt einem eindeutigen Muster:

1. Eingabeszenario wird erfasst und validiert.
2. Simulationskern berechnet den Ablauf für ein Verfahren.
3. Ergebnisstrukturen (Ereignisse, Segmente, Snapshots, Metriken) werden bereitgestellt.
4. Darstellungsschicht rendert die aktuelle Sicht und reagiert auf Interaktionsbefehle.

Entscheidend ist, dass Visualisierung und Kennzahlen auf denselben Ergebnisstrukturen basieren. Dadurch entstehen keine widersprüchlichen Anzeigen zwischen Ablaufdarstellung und numerischer Bewertung.

4.5. Interaktionsmodell für den Lernprozess

Das Interaktionsmodell folgt einem didaktisch motivierten Nutzungsfluss:

1. Szenario definieren,
2. Verfahren auswählen,
3. Ablauf steuern und beobachten,
4. Ergebnisse interpretieren,
5. Verfahren vergleichen.

Die Interaktion ist damit kein rein technischer Bedienvorgang, sondern Teil des Lernprozesses. Die Steuerung beeinflusst die Sicht auf den Ablauf (z. B. Schrittmodus), ohne die fachliche Berechnung selbst zu verändern.

4.6. HCI-Leitlinien im Entwurf

Für eine Lernanwendung ist die UI ein fachlich relevantes Element. Der Entwurf folgt daher grundlegenden HCI-Leitlinien:

- zentrale Zustände müssen direkt sichtbar sein,
- wichtige Ereignisse (Präemption, Kontextwechsel, Abschluss) müssen klar markiert werden,
- Bedienung und Analyse sollen in einem konsistenten Ablauf zusammenwirken,
- die Oberfläche soll Orientierung bieten und nicht zusätzliche kognitive Last erzeugen.

Damit wird sichergestellt, dass die Anwendung nicht nur technisch korrekt arbeitet, sondern auch als didaktisches Werkzeug nutzbar ist.

4.7. Erweiterbarkeit und Entwurfsgrenzen

Die Architektur ist auf Erweiterbarkeit ausgelegt: Neue Scheduling-Verfahren können als zusätzliche Strategien im Simulationskern ergänzt werden, ohne die UI grundlegend umzubauen. Ebenso lassen sich neue Kennzahlen oder Ansichten integrieren, sofern sie auf den vorhandenen Ergebnisstrukturen aufsetzen.

Eine bewusste Entwurfsgrenze liegt in der Fokussierung auf präemptive Verfahren und den

Lehrkontext. Die Architektur ist zwar übertragbar, aber nicht als vollständiger Ersatz für produktive Betriebssystem-Scheduler konzipiert.

5. Implementierung

Die Implementierung setzt den in Kapitel 4 beschriebenen Entwurf in eine lauffähige Webanwendung um. Ziel ist die technisch saubere Realisierung eines Visualizers, der präemptive Scheduling-Verfahren fachlich korrekt berechnet und gleichzeitig verständlich darstellt.

5.1. Technische Grundlagen und verwendete Technologien

Die Anwendung wurde als clientseitige Webanwendung mit Vue 3 und TypeScript umgesetzt. Vue.js Core Team. Vue strukturiert die Benutzeroberfläche komponentenbasiert, während TypeScript die fachlichen Datenmodelle und Zustände typensicher abbildet. Der Build- und Entwicklungsprozess erfolgt mit Vite, die automatisierte Verifikation mit Vitest. Für gezielte Übergänge in der Oberfläche wird GSAP eingesetzt. Green-Sock.

Die zentrale technische Entscheidung ist die Trennung von Simulationslogik, Zustandsverwaltung und Darstellung. Dadurch bleibt der Simulationskern unabhängig testbar, während die UI konsistente Daten visualisiert.

5.2. Datenmodell und Simulationskern

Die fachliche Basis bildet ein gemeinsames Task-Modell mit Attributen wie Ankunftszeit, Burst-Zeit, Priorität, Restzeit und Status. Auf dieser Grundlage werden alle Verfahren innerhalb eines einheitlichen Simulationsrahmens berechnet.

Ein Simulationslauf erzeugt mehrere zusammenhängende Ergebnisstrukturen:

- **Ereignislog:** chronologische Ereignisse (z. B. Dispatch, Präemption, Abschluss),
- **Segmente:** zeitlich abgegrenzte Ausführungsintervalle für die Gantt-Darstellung,
- **Snapshots:** Zustandsabbilder für Wiedergabe und Zeitsprung,
- **Metriken:** aggregierte Kennzahlen wie Warte-, Durchlauf- und Reaktionszeit sowie Kontextwechsel.

Diese Struktur stellt sicher, dass Ablaufdarstellung und Kennzahlen aus derselben Berechnungsbasis stammen und somit konsistent bleiben.

5.3. Umsetzung der Scheduling-Verfahren

Die Verfahren Round Robin, LCFS, Strict Priority und MLFQ wurden als präemptive Strategien innerhalb derselben Simulationslogik implementiert.

- **Round Robin:** Präemption nach Ablauf eines konfigurierbaren Zeitquantums.
- **LCFS (präemptiv):** Neu eintreffende Prozesse verdrängen den laufenden Prozess.
- **Strict Priority:** Auswahl des jeweils höchstpriorisierten bereiten Prozesses mit sofortiger Präemption bei höher priorisierten Neuankünften.
- **MLFQ:** Mehrere Warteschlangen mit stufenabhängigen Quantumswerten und Herabstufungslogik.

Alle Verfahren verwenden dieselbe Tick- und Ereignisverarbeitung. Unterschiede in Kennzahlen und Ablauf ergeben sich daher aus der Verfahrensstrategie und nicht aus abweichender Berechnungsinfrastruktur.

5.4. Umsetzung der Visualisierung

Die Visualisierung ist in klar getrennte Funktionsbereiche gegliedert: Eingabe- und Konfigurationsbereich, Laufsteuerung, Fokusansicht, Kennzahlenbereich und Vergleichsansicht.

Die zeitliche Ausführung wird in einer Gantt-Darstellung auf Basis der Segmentdaten visualisiert. Ergänzende Ansichten zeigen aktuelle Wartesituationen und laufzeitbezogene Metriken. Die Umsetzung nutzt SVG-basierte Darstellungen, um Zeitachsen, Segmentgrenzen und Ereignisse präzise abzubilden.

Ziel der Visualisierung ist nicht dekorative Darstellung, sondern fachliche Lesbarkeit: Entscheidungen des Schedulers sollen zeitlich und inhaltlich nachvollziehbar werden.

5.5. Interaktionsmechanismen und Nutzerworkflow

Die Interaktion folgt einem didaktisch motivierten Ablauf: Szenario definieren, Verfahren auswählen, Lauf steuern, Ergebnis interpretieren und Verfahren vergleichen.

Die Wiedergabelogik arbeitet auf vorberechneten Snapshots und startet die Simulation nicht bei jedem Interaktionsschritt neu. Dadurch bleiben Steuerfunktionen wie Start, Pause, Schrittmodus, Geschwindigkeitsanpassung und Zeitsprung reaktiv, ohne die fachliche Konsistenz zu gefährden.

Für den Vergleichsmodus werden mehrere Läufe auf derselben Eingabebasis nebeneinander dargestellt. Dies unterstützt die direkte Gegenüberstellung algorithmischer Unterschiede.

5.6. Build- und Deploymentprozess

Für die Bereitstellung wurde ein containerbasierter Deploymentprozess verwendet. Die Anwendung wird über den Vite-Build in statische Artefakte überführt und in einem Docker-Image paketierte. Damit entsteht eine reproduzierbare Laufzeitumgebung unabhängig vom lokalen Entwicklungsrechner.

Das Deployment erfolgt auf einer Hosting-Plattform mit Container-Unterstützung. Änderungen am Quellcode können dadurch konsistent gebaut und in einer stabilen Umgebung veröffentlicht werden. Für den Einsatz im Lehrkontext ist diese Reproduzierbarkeit wesentlich, da dieselbe Version für Demonstration und Nutzung bereitsteht.

5.7. Besondere technische Herausforderungen

Eine zentrale Herausforderung war die Synchronisation von Simulationsgenauigkeit und interaktiver Bedienung. Präemptive Verfahren erzeugen häufige Zustandswechsel; gleichzeitig muss die Oberfläche unmittelbar auf Nutzerinteraktionen reagieren.

Ein weiterer Schwerpunkt war die Sicherung der Deterministik. Dafür wurden klare Zustandsübergänge, konsistente Datenmodelle und testbare Schnittstellen priorisiert. Zusätzlich entstand ein technischer Zielkonflikt zwischen Detailtiefe und Ressourcenbedarf: Dichtere Snapshot-Folgen verbessern Analyse und Zeitsprung, erhöhen jedoch Speicher- und Verarbeitungsaufwand. Dieser Zielkonflikt wurde durch parametrisierbare Granularität der Wiedergabedaten adressiert.

6. Qualitätssicherung

Für ein didaktisches Werkzeug mit fachlicher Relevanz ist Qualitätssicherung ein zentraler Bestandteil der Umsetzung. Entscheidend sind nicht nur technische Funktionsfähigkeit, sondern insbesondere fachliche Korrektheit der Simulation, reproduzierbares Verhalten und nachvollziehbare Interaktion.

6.1. Qualitätsziele und Prüfstrategie

Die Qualitätssicherung verfolgt drei Hauptziele:

- **Fachliche Korrektheit:** Scheduling-Entscheidungen und Kennzahlen müssen inhaltlich konsistent sein.
- **Reproduzierbarkeit:** Identische Eingaben müssen identische Abläufe erzeugen.
- **Bedienbarkeit und Zugänglichkeit:** Zentrale Funktionen müssen verständlich und verlässlich nutzbar sein.

Zur Erreichung dieser Ziele wurde ein mehrstufiger Ansatz gewählt: automatisierte Tests der Kernlogik, Prüfungen zentraler Interaktionspfade sowie Bewertung von Accessibility-Aspekten in der Oberfläche.

6.2. Automatisierte Verifikation der Simulationslogik

Die Simulationslogik wird mit Vitest automatisiert geprüft Vitest Team. Im Fokus stehen deterministische Berechnungsschritte, korrekte Präemptionsentscheidungen und konsistente Ableitung der Kennzahlen.

Getestet werden unter anderem:

- Quantum-Verhalten und Wiederanstellen bei Round Robin,
- Präemption bei Neuankunft in präemptiven Verfahren (z. B. LCFS, Strict Priority),
- Herabstufungslogik in MLFQ,
- Konsistenz von Ereignisfolge, Segmenten und Endzustand.

Zusätzlich wird geprüft, ob ungültige oder unvollständige Eingabedaten korrekt normalisiert werden und ob erzeugte Segmente gültige Zeitgrenzen aufweisen.

6.3. Prüfung von Interaktion und Accessibility

Neben der Fachlogik wurde die Oberfläche hinsichtlich Bedienbarkeit und Zugänglichkeit betrachtet. Dazu zählen semantische Struktur, nachvollziehbare Fokusführung und konsistente Interaktionsabläufe. Orientierung bieten die Grundprinzipien der WCAG W3C [2018].

Für den Lehrkontext ist wesentlich, dass Nutzende ohne unnötige Hürden zwischen Szenariokonfiguration, Ablaufsteuerung und Ergebnisinterpretation wechseln können. Klare Statusanzeigen und eindeutige Steuerzustände unterstützen diesen Prozess.

6.4. Reproduzierbarkeit und Vergleichbarkeit

Ein zentrales Qualitätsmerkmal ist die deterministische Reproduzierbarkeit: Bei identischem Szenario und identischen Parametern muss derselbe Ablauf mit denselben Kennzahlen entstehen. Diese Eigenschaft ist für didaktische Vergleiche essenziell, da nur so Unterschiede zwischen Verfahren sachlich auf die jeweilige Scheduling-Strategie zurückgeführt werden können.

Auch die Vergleichsansicht profitiert davon: Mehrere Läufe lassen sich auf gemeinsamer Eingabebasis direkt gegenüberstellen, ohne dass zufällige Abweichungen die Interpretation verfälschen.

6.5. Erreichte Testabdeckung und Grenzen

Zum Zeitpunkt der Erstellung umfasst die automatisierte Testsuite elf Testdateien mit insgesamt 34 erfolgreichen Tests. Die Tests decken zentrale Kern- und Integrationspfade ab und erhöhen die technische Verlässlichkeit der Implementierung deutlich.

Die Testsuite deckt damit Kernlogik und zentrale Integrationspfade ab, ist jedoch nicht als vollständiger formaler Korrektheitsbeweis aller möglichen Prozesskonstellationen zu verstehen.

Gleichzeitig bestehen Grenzen: Die Testsuite stellt keine vollständige Abdeckung aller theoretisch möglichen Prozesskonstellationen dar und ersetzt keine empirische Untersuchung der Lernwirksamkeit mit Nutzergruppen. Die Qualitätssicherung ist daher als risikoorientierte Absicherung der Kernfunktionen zu verstehen.

6.6. Fazit zur Qualitätssicherung

Die Qualitätssicherung wurde nicht als nachgelagerte Kontrolle, sondern als integraler Bestandteil der Entwicklung umgesetzt. Die Kombination aus deterministischer Simulati-

onslogik, automatisierter Verifikation und systematischer Interaktionsprüfung schafft die Grundlage für ein fachlich belastbares und didaktisch nutzbares Lehrwerkzeug.

7. Ergebnis und Produktpräsentation

Dieses Kapitel stellt das entwickelte Endprodukt vor und ordnet es im Hinblick auf die in den vorherigen Kapiteln formulierten Ziele ein. Im Mittelpunkt steht die fachlich belastbare und visuell nachvollziehbare Produktwirkung der Anwendung als didaktisches Werkzeug.

7.1. Entwickeltes Endprodukt

Das Ergebnis der Arbeit ist eine interaktive Webanwendung zur Visualisierung präemptiver CPU-Scheduling-Verfahren. Die Anwendung ermöglicht es, Prozesse mit relevanten Parametern zu definieren, verschiedene Verfahren auszuführen und deren Ablauf sowie Kennzahlen vergleichend zu analysieren.

Damit verbindet das Produkt drei zentrale Aspekte:

- fachlich konsistente Simulationslogik,
- verständliche, interaktive Darstellung zeitlicher Abläufe,
- direkte Vergleichbarkeit mehrerer Verfahren auf identischer Datenbasis.

Die Anwendung ist damit nicht nur eine prototypische Demonstration, sondern ein nutzbares Werkzeug für Lehr- und Lernszenarien.

7.2. Produktüberblick und Nutzerfluss

Der typische Arbeitsablauf in der Anwendung folgt einer klaren Reihenfolge:

1. Definition oder Auswahl eines Szenarios,
2. Auswahl eines Scheduling-Verfahrens und Parametrisierung,
3. Ausführung und Steuerung des Laufs,
4. Interpretation von Ablauf und Kennzahlen,
5. Vergleich mit alternativen Verfahren.

Dieser Ablauf unterstützt das didaktische Ziel, algorithmische Entscheidungen nicht isoliert als Endwerte, sondern im zeitlichen Kontext ihrer Entstehung zu verstehen.

7.3. Produktpräsentation mit Screenshots

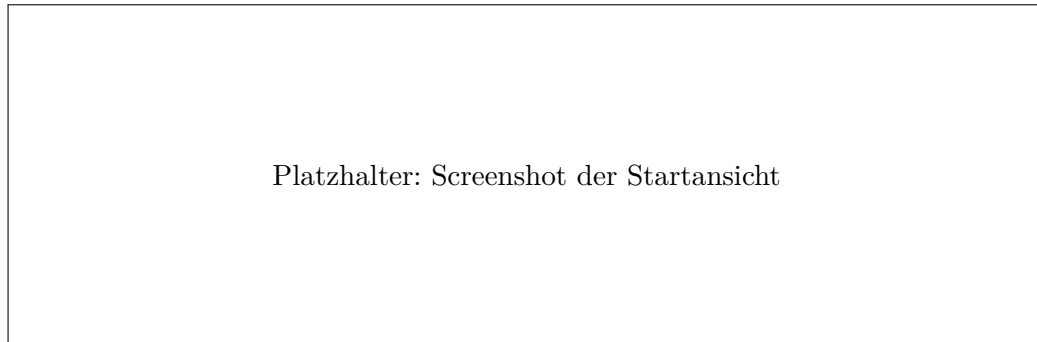


Abbildung 7.1.: Startansicht der Anwendung mit Ausgangszustand und Einstieg in die Szenario-konfiguration.

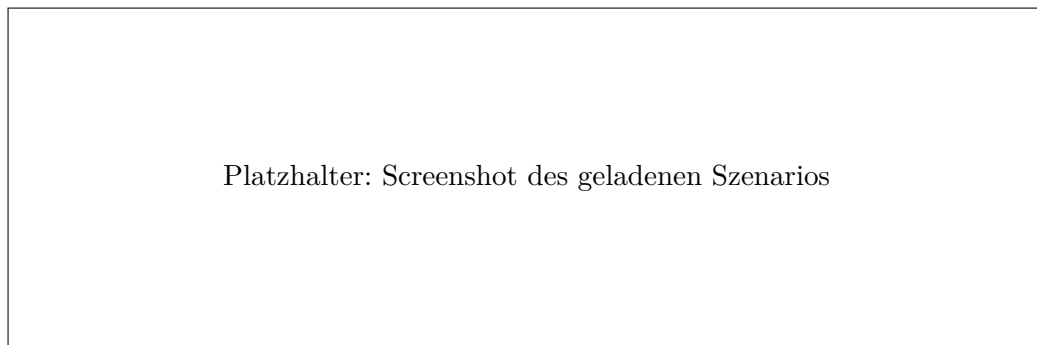


Abbildung 7.2.: Geladenes Szenario mit sichtbaren Task-Parametern als Grundlage für die Simulation.

7.4. Umgesetzte Funktionalität im Soll-Ist-Abgleich

Die im Verlauf der Arbeit abgeleiteten Anforderungen wurden in zentralen Funktionsbereichen umgesetzt:

- **Szenarioverwaltung:** Erfassung und Bearbeitung von Task-Daten mit relevanten Scheduling-Parametern.
- **Verfahrensauswahl:** Ausführung von Round Robin, LCFS, Strict Priority und MLFQ innerhalb eines einheitlichen Simulationsrahmens.
- **Ablaufsteuerung:** Start, Pause, Schrittmodus, Zeitsprung und Geschwindigkeitsanpassung.
- **Visualisierung:** Zeitliche Darstellung der Ausführung mit Ereignisbezug.
- **Kennzahlenanalyse:** Ausgabe zentraler Metriken pro Lauf.
- **Vergleichsmodus:** Gegenüberstellung mehrerer Läufe unter identischen Eingaben.

Platzhalter: Screenshot der Fokusansicht

Abbildung 7.3.: Fokusansicht mit zeitlicher Ablaufdarstellung und Kennzahlenbereich für einen einzelnen Lauf.

Platzhalter: Detailansicht des Gantt-Diagramms

Abbildung 7.4.: Detaildarstellung der Gantt-Ansicht mit segmentierter Ausführung und sichtbaren Task-Wechseln.

Platzhalter: Warteschlangen- und Statusdarstellung

Abbildung 7.5.: Warteschlangen- und Statusdarstellung zur Einordnung der aktuellen Scheduling-Entscheidung.

Platzhalter: Detailansicht der Kennzahlen

Abbildung 7.6.: Kennzahlenbereich mit Wartezeit, Durchlaufzeit und Reaktionszeit als quantitative Auswertung des Laufs.



Platzhalter: Vergleichsansicht mehrerer Verfahren

Abbildung 7.7.: Vergleichsansicht mehrerer Verfahren auf identischer Szenariobasis.

Damit wird der in Kapitel 3 formulierte funktionale Kern vollständig adressiert. Nicht-funktionale Ziele wie Deterministik, Nachvollziehbarkeit und Erweiterbarkeit wurden durch Architektur und Implementierung ebenfalls unterstützt.

7.5. Ergebnisbetrachtung

Das entwickelte Produkt erfüllt die Zielsetzung der Arbeit: Präemptive Scheduling-Verfahren werden nicht nur berechnet, sondern in ihrer Dynamik verständlich darstellbar und vergleichbar gemacht. Die Verbindung aus Ablaufvisualisierung, Kennzahlen und Interaktion ermöglicht es, algorithmische Unterschiede in einem didaktisch nutzbaren Kontext zu analysieren.

Besonders relevant ist dabei, dass Entscheidungspfade sichtbar werden. Nutzende sehen nicht nur, *welches* Ergebnis entsteht, sondern auch *wie* es zustande kommt. Dadurch entsteht ein praktischer Mehrwert für Lehre, Demonstration und eigenständiges Lernen.

8. Fazit und Ausblick

Die vorliegende Arbeit zeigt, dass sich präemptive CPU-Scheduling-Verfahren in Form einer interaktiven Webanwendung fachlich konsistent und didaktisch nachvollziehbar vermitteln lassen. Die zentrale Leistung liegt in der Verbindung von Simulationslogik, Visualisierung und steuerbarer Interaktion zu einem zusammenhängenden Lehrwerkzeug.

Ausgehend von den theoretischen Grundlagen wurden Anforderungen an Fachlogik, Usability und Transparenz systematisch abgeleitet und in ein umsetzbares Architekturmodell überführt. Die Implementierung realisiert diese Anforderungen in einer modularen Struktur, in der Simulationskern und Darstellung klar getrennt sind. Dadurch bleiben Berechnungen nachvollziehbar, testbar und unter identischen Bedingungen reproduzierbar.

Die Anwendung erfüllt damit die Zielsetzung der Arbeit: Sie macht Scheduling-Entscheidungen sichtbar, verknüpft den zeitlichen Ablauf mit relevanten Kennzahlen und ermöglicht die direkte Gegenüberstellung mehrerer Verfahren auf gemeinsamer Szenariobasis. Der didaktische Mehrwert entsteht insbesondere dadurch, dass nicht nur Endergebnisse präsentiert werden, sondern auch die zugrunde liegenden Entscheidungspfade.

Die Qualitätssicherung stützt diese Aussage durch automatisierte Tests der Kernlogik, Prüfungen zentraler Interaktionspfade und die Berücksichtigung grundlegender Accessibility-Anforderungen. Damit ist die Anwendung sowohl als technische Umsetzung als auch als Lerninstrument belastbar einsetzbar.

8.1. Ausblick

Für eine Weiterentwicklung bieten sich mehrere Richtungen an. Erstens kann die fachliche Breite durch zusätzliche Scheduling-Verfahren und differenziertere Lastmodelle erweitert werden. Zweitens lässt sich die Analyse vertiefen, etwa durch weitere Kennzahlen, Exportfunktionen und ergänzende Vergleichsmodi. Drittens ist eine didaktische Erweiterung denkbar, beispielsweise durch geführte Lernpfade, Aufgabenserien und vorbereitete Szenariosets für den Einsatz in Lehrveranstaltungen.

Darüber hinaus wäre eine empirische Evaluation mit Studierenden ein sinnvoller nächster Schritt, um den tatsächlichen Lerneffekt systematisch zu untersuchen. Die vorliegende Arbeit schafft hierfür eine tragfähige technische und konzeptionelle Grundlage.

Literaturverzeichnis

GreenSock. Gsap documentation. URL <https://greensock.com/docs/>.

Christopher D. Hundhausen, Sarah A. Douglas, and John T. Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages & Computing*, 13(3): 259–290, 2002. doi: 10.1006/jvlc.2002.0237.

L. W. C. Meing and D. R. Tandingan. Exploring multilevel feedback queue combinations and regression-based time quanta in scheduling algorithms. *Journal of Information Systems Engineering and Management*, 2025. URL <https://jisem-journal.com/index.php/journal/article/view/699>.

William Stallings. *Betriebssysteme: Prinzipien und Umsetzung*. Pearson Studium, München, 4., überarb. auflage edition, 2003.

Andrew S. Tanenbaum and Herbert Bos. *Moderne Betriebssysteme*. Pearson Studium, München, 4., aktualisierte auflage edition, 2016.

Vite. Vite documentation. URL <https://vitejs.dev/>.

Vitest Team. Vitest documentation. URL <https://vitest.dev/>.

Vue.js Core Team. Vue 3 documentation. URL <https://vuejs.org/>.

W3C. Web content accessibility guidelines (wcag) 2.1, 2018. URL <https://www.w3.org/TR/WCAG21/>.

A. Anhang

A.1. Ergänzende Abbildungen

A.2. Zusätzliche Testfälle und Szenarien

A.3. Ergänzende technische Artefakte (z. B. Konfigurationsbeispiele)

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich diese BACHELORARBEIT selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, alle Ausführungen, die anderen Schriften wörtlich oder sinngemäß entnommen wurden, kenntlich gemacht sind und die Arbeit in gleicher oder ähnlicher Fassung noch nicht Bestandteil einer Studien- oder Prüfungsleistung war.

Marten Kirschenhofer